The req and res objects: reading and responding

9 min read · Updated 24 Jul 2026 · Topic 38 of 70

Bookmark · Mark done

Why it matters

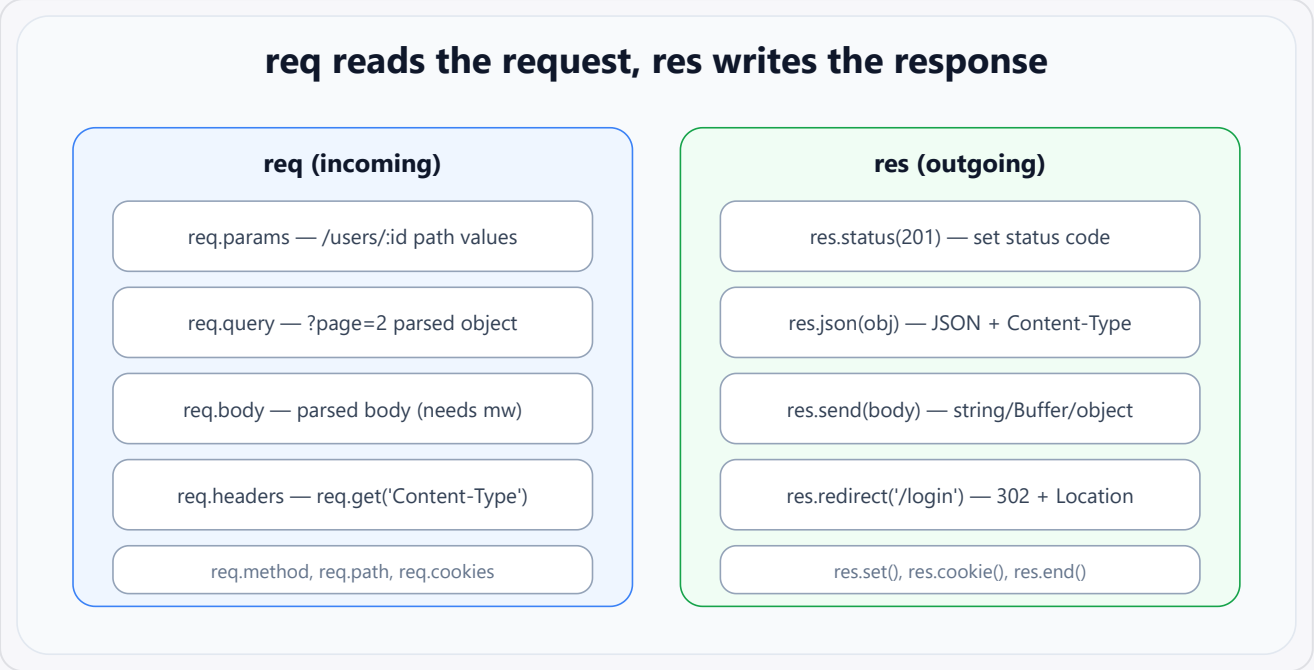
Every Express handler you will ever write does two things: it *reads* the incoming request and it *writes* a response. The `req` and `res` objects are the interface for both, and knowing them fluently is the difference between fumbling through docs on every endpoint and writing handlers on autopilot. Interviewers use these objects as a fast competence check: "where does the request body come from?", "how do you set a 201 with JSON?", "what does `res.json()` do that `res.send()` doesn't?" — quick questions that instantly reveal whether you have actually built APIs or just skimmed a tutorial.

Beyond the individual methods, these objects teach an important truth about Express: they are Node's own `IncomingMessage` and `ServerResponse`, *extended* with conveniences. That means everything you know about Node HTTP still applies, and the Express helpers are sugar on top. This topic maps the request side (`params`, `query`, `body`, `headers` and how body parsing actually works) and the response side (`json`, `status`, `send`, `redirect`, headers), plus the gotchas — the string-typing, the one-response rule, the parsing prerequisites — that turn into production bugs when ignored.

The concept, simply

`req` **is the request: everything the client sent.** It bundles four sources of input. `req.params` holds path parameters (`/users/:id` → `req.params.id`). `req.query` holds the parsed query string (`?page=2` → `req.query.page`). `req.body` holds the parsed request body (the JSON a client POSTs) — but only if a body-parsing middleware ran first. `req.headers` (and the helper `req.get('Header-Name')`) holds the request

`res` **is the response: everything you send back.** Its key methods are `res.status(code)` to set the HTTP status (chainable), `res.json(obj)` to send a JSON body (it serialises and sets `Content-Type: application/json`), `res.send(body)` to send a string, Buffer, or object (it guesses the type), and `res.redirect(url)` to send a redirect. You also set headers with `res.set()` / `res.header()`, cookies with `res.cookie()`, and end a bodiless response with `res.status(204).end()`. The golden rule: **you send exactly one response per request.**



req is the read side (params, query, body, headers); res is the write side (status, json, send, redirect). One response per request.

The one concept that trips everyone: **body parsing is not automatic**. When a client POSTs JSON, the body arrives as a raw stream of bytes. Express does not decode it into `req.body` unless you registered `express.json()` (for JSON) or `express.urlencoded()` (for HTML-form encoding) earlier in the pipeline. Without that, `req.body` is `undefined`. Params and query are parsed for free; the body is opt-in because parsing strategy depends on content type.

How it works

methods mixed in. So `res.json()` ultimately calls `res.end()` underneath, and `req.headers` is Node's raw headers map. This layering is why you can always escape to the lower level: `res.setHeader()`, `res.writeHead()`, and `res.end()` all still work. Express helpers exist to remove repetitive steps, not to hide the machine.

Body parsing works because a parser is a middleware that reads the request stream to completion, decodes it according to the `Content-Type` header, and assigns the result to `req.body` before calling `next()`. `express.json()` only acts on requests whose `Content-Type` is `application/json`; `express.urlencoded()` handles `application/x-www-form-urlencoded`. If the incoming type does not match, the parser passes through and `req.body` stays as the default (an empty object in recent Express, historically `undefined`). Both parsers accept a `limit` option to cap body size — an important guard against memory-exhaustion attacks. File uploads (`multipart/form-data`) need a dedicated library like `multer`, not the built-in parsers.

On the response side, `res.json(value)` runs `JSON.stringify` on the value, sets the `Content-Type` to `application/json`, and sends it. `res.send(value)` is more flexible and infers the type: a string is sent as HTML/text, a Buffer as binary, and — importantly — an object or array is sent as JSON (it delegates to `res.json`). For a JSON API, prefer `res.json()` explicitly; it is unambiguous. `res.status(code)` only *sets* the status and returns `res` so you can chain (`res.status(201).json( ... )`); it does not send anything on its own. `res.redirect(url)` defaults to a 302 and sets the `Location` header; pass a status first for a permanent redirect (`res.redirect(301, url)`).

The strict rule underneath all of this is that **an HTTP response can be sent only once**. Methods like `res.json`, `res.send`, `res.redirect`, and `res.end` *finish* the response. Calling any of them a second time — or calling one after the headers have been flushed — throws `ERR_HTTP_HEADERS_SENT` ("Cannot set headers after they are sent"). This is why you `return` after responding, and why a handler that responds must not also fall through to `next()` into code that responds again. Types are the other recurring gotcha: values in `req.params` and `req.query` are always strings, and `req.body` reflects exactly what the client sent — never trust it without validation.

Code examples

JAVASCRIPT Copy

```
// POST /users/42/notes?draft=true  with a JSON body
app.post('/users/:id/notes', (req, res) => {
  const userId = req.params.id;             // '42' (string, from the path)
  const isDraft = req.query.draft;          // 'true' (string, from ?draft=
  const note = req.body;                    // { text: '...' } (needs expre
  const auth = req.get('Authorization');    // header value or undefined
  res.status(201).json({ userId, isDraft, note });
});
```

2. Enabling body parsing — the prerequisite for `req.body`.

JAVASCRIPT Copy

```
app.use(express.json({ limit: '1mb' }));            // JSON bodies → req.
app.use(express.urlencoded({ extended: true }));    // HTML form bodies →

app.post('/login', (req, res) => {
  const { email, password } = req.body;    // undefined without the parsers
  res.json({ email });
});
```

3. Response helpers — status, json, send, redirect, headers.

JAVASCRIPT Copy

```
app.get('/created', (req, res) => res.status(201).json({ id: 1 }));    // J

app.get('/text', (req, res) => res.send('plain string'));              // te

app.get('/nothing', (req, res) => res.status(204).end());              // no

app.get('/old', (req, res) => res.redirect('/new'));                   // 30
app.get('/moved', (req, res) => res.redirect(301, '/permanent'));      // 30

app.get('/download', (req, res) => {
```

```
});
```

4. The one-response rule in action — and how to obey it.

JAVASCRIPT Copy

```
app.get('/guard/:id', (req, res) => {
  if (!req.params.id) {
    return res.status(400).json({ error: 'id required' }); // return = sto
  }
  // without the return above, execution would continue and try to respond
  res.json({ id: req.params.id });
});
```

Common mistakes & gotchas

- `req.body` **is** `undefined`**.** You forgot `express.json()` / `express.urlencoded()`, or registered it after the route, or the client didn't send a matching `Content-Type`.
- **Sending two responses.** Calling `res.json()` then `res.send()` (or responding without `return` and falling through) throws "Cannot set headers after they are sent". Respond once; `return` after.
- **Assuming numbers.** `req.params.id` and `req.query.page` are strings; do `Number( ... )` and validate. `?flag=false` is the truthy string `"false"`, not a boolean.
- **Trusting** `req.body`**.** It is whatever the client sent, including missing or malicious fields. Validate/whitelist before using it (schema library or manual checks).
- **Using** `express.json()` **for file uploads.** Multipart uploads need `multer`; the JSON parser won't touch `multipart/form-data`.
- **Forgetting a body-size** `limit`**.** An unbounded parser lets a client send a huge body and exhaust memory; set `limit`.
- **Confusing** `req.path`**,** `req.url`**, and** `req.originalUrl`**.** Inside a mounted router `req.url` is relative to the mount; `req.originalUrl` is the full path the client

Best practices

- **Register body parsers once, near the top**, with an explicit `limit`, before any route that reads `req.body`.
- **Prefer `res.json()` for APIs** over `res.send()` — it is explicit about content type and intent, even though `send` would also JSON-encode an object.
- **Always set an intentional status** — 201 for creates, 204 for empty successes, 4xx for client errors — instead of letting everything default to 200.
- **Validate and coerce request inputs at the boundary** (params, query, body) with a schema library, so handlers work with clean, typed data.
- **Always `return` after sending a response** to guarantee the one-response rule and avoid header-already-sent errors.
- **Read headers via `req.get('Name')`** (case-insensitive) rather than indexing `req.headers` with a guessed casing.

⭐ Interview lens

Q: "Where do `req.params`, `req.query`, and `req.body` each come from?"
A: `req.params` from named segments in the route path (`/users/:id`); `req.query` from the query string after the `?`; `req.body` from the parsed request body — but only after a body-parsing middleware like `express.json()` runs. Params and query are parsed automatically; the body is opt-in.

Q: "Why is `req.body` undefined even though the client sent JSON?"
A: No body parser ran before the handler. You need `app.use(express.json())` registered above the route, and the client must send `Content-Type: application/json` so the parser activates.

A: `res.json()` always serialises the argument with `JSON.stringify` and sets `Content-Type: application/json`. `res.send()` infers the type — string as text/HTML, Buffer as binary, object/array delegated to `res.json()`. For a JSON API, use `res.json()` for clarity.

Q: "How do you send a 201 with a JSON body?"
A: `res.status(201).json(data)`. `res.status()` only sets the code and returns `res` for chaining; `res.json()` serialises and sends. It does not send by itself.

Q (trap): "What throws 'Cannot set headers after they are sent'?"
A: Sending the response more than once — e.g. calling `res.json()` and then another `res.send()` / `res.redirect()`, or responding and then falling through into `next()` to code that also responds. An HTTP response can be finished only once; `return` after you respond.

Q: "Are `req.params` and `req.query` values typed?"
A: No — always strings (query values can also be arrays/objects from repeated or bracketed keys, but never numbers or booleans). `?active=false` gives the truthy string `"false"`. You must coerce and validate.

Cheat-card

- `req` reads: `req.params` (path), `req.query` (`?`), `req.body` (needs parser), `req.get('H')` (headers).
- `res` writes: `res.status(n)` (chainable), `res.json(obj)`, `res.send(x)`, `res.redirect([code,] url)`.
- `req.body` needs `express.json()` / `express.urlencoded()` **before** the route.
- Params & query are **always strings**. Coerce & validate.
- **One response per request.** `return` after responding.

- `req` / `res` extend Node's `IncomingMessage` / `ServerResponse` — `res.end()` / `res.setHeader()` still work.

QUICK CHECK — no marks, just for you.

1. What is the difference between res.send() and res.json()?

- ◯ res.send() always sends JSON; res.json() always sends plain text
- ◯ res.json() cannot set a status code, but res.send() can
- ◯ They are identical aliases with no behavioural difference
- ◯ res.json() always runs JSON.stringify and sets Content-Type: application/json; res.send() infers the type (string as text, Buffer as binary, object delegated to res.json)

Check answer

2. How do you send a 201 status with a JSON body in Express?

- ◯ res.status(201).json(data) — res.status() only sets the code and returns res for chaining; res.json() serialises and sends
- ◯ res.json(201, data), passing the status as the first argument
- ◯ res.send(201).json(data), since send sets the status
- ◯ res.status(201) alone, which sends an empty JSON body

Check answer

3. What causes "Cannot set headers after they are sent to the client"?

Setting a header before calling res.status()

- Sending the response more than once — e.g. calling res.json() then res.send()/res.redirect(), or responding and then falling through to code that responds again
- Calling res.json() with an object larger than the body-size limit
- Reading req.body after the response has started

Check answer

Previous
Error handling: the 4-arg handler and async errors

Next
REST Principles: Constraints, Resources & Statelessness

LEARN MORE

ARTICLES & DOCS

- Request Object (req)
  expressjs.com
- Response Object (res)
  expressjs.com

VIDEOS



Node.js and Express.js - Full Course
freeCodeCamp.org

Hand-picked references for this lesson.